

LRM-GPU: Alleviating Synchronization Overhead for Multi-Chiplet GPU Architecture

Baiqing Zhong[†], Zhirong Ye[†], Xiaojie Li[†], Peilin Wang[†], Haiqiu Huang[†], Zhaolin Li[‡],
Zhiyi Yu[†], and Mingyu Wang^{†,*}

[†]*School of Microelectronics Science and Technology, Sun Yat-Sen University, China*

[‡]*Department of Computer Science and Technology, Tsinghua University, China*

Email(s): {wangmingyu, yuzhiyi}@mail.sysu.edu.cn, lz173@mail.tsinghua.edu.cn,

{zhongbq, yezhr6, lixj263, wangplin, huanghq73}@mail2.sysu.edu.cn

Abstract—With the slowdown of process scaling and the advancement of packaging technologies, multi-chiplet GPUs have emerged as a highly promising architecture to improve the scalability of GPU performance further. Moreover, requiring adherence to atomicity and memory consistency models for shared data efficient synchronization is crucial to leverage the performance advantages of the multi-chiplet GPU architecture. However, the memory systems of multi-chiplet GPUs introduce deeper cache hierarchies and increased non-uniformity, both of which significantly exacerbate the overhead of synchronization. Specifically, acquire/release synchronization operations should invalidate/flush caches, an overhead that is significantly increased by the presence of additional cache level, and atomic operations for synchronization performed across chiplets are further impacted by the limited bandwidth of inter-chiplet links.

To address these challenges, this paper proposes LRM-GPU to provide efficient synchronization support for multi-chiplet GPUs. In order to reduce the overhead caused by the additional cache level, LRM-GPU leverages lazy release consistency in multi-chiplet GPUs, whereby the additional level of cache only performs coherence actions when the ownership of synchronization variables changes between different chiplets. LRM-GPU also implements a directory in the last-level cache to track the synchronization variables. To mitigate the overhead of atomic operations for inter-chiplet synchronization under limited inter-chiplet bandwidth, LRM-GPU proposes an in-network synchronization atomic merging unit to merge atomic requests across chiplets, thereby reducing the inter-chiplet synchronization traffic of atomic operations. Experimental evaluation demonstrates that, compared with the MCM-GPU, LRM-GPU achieves an average speedup of 1.33 $\times$. Moreover, compared with the state-of-the-art work HMG, it also achieves the speedup of 1.22 $\times$, reduces 52% of inter-chiplet traffic, and reduces 32% of energy consumption on average.

I. INTRODUCTION

Modern graphics processing units (GPUs) leverage the Single-Instruction-Multiple-Threads (SIMT) architecture to efficiently execute multiple concurrent threads, proving instrumental in diverse computational domains such as high-performance computing (HPC) and machine learning [6, 26, 35, 46, 50]. As the computational demands continue to increase, GPUs have leveraged shrinking process features to dramatically increase transistor density, while also manufacturing ever-larger dies to meet the strong scaling requirements of applications [30, 34, 36, 37]. However, as transistor scaling

has slowed down and manufacture constraints the maximum die size, the scalability of GPU performance has become increasingly constrained [22, 23, 39]. These challenges have motivated researchers to explore alternative approaches to enhance the performance scalability of GPUs. Recent research has explored the composition of multiple smaller chips into a single large-scale, aggregated system, an approach commonly referred to as a Multi-Chip Module (MCM) or chiplet [2, 3, 15, 45]. The multi-chiplet GPU architecture distributes computational workloads across multiple GPU chiplets to enable scalable and efficient GPU design.

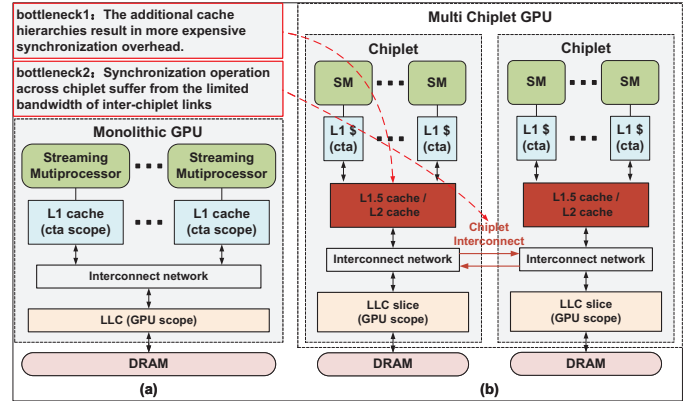


Fig. 1. Current GPU Architecture: (a) Monolithic GPU; (b) Multi-Chiplet GPU

However, as GPUs become increasingly general-purpose, they are also used for applications that require efficient support for more data sharing. Therefore, the critical aspect of showcasing multi-chiplet GPU advantages hinges on the strict adherence of all threads to efficient synchronization mechanisms [9, 27]. These mechanisms demand that shared data updates adhere to consistent access sequences and update protocols (i.e., ensuring consistency), while effectively managing conflicting accesses to uphold the deterministic nature of multi-threaded program execution (i.e., ensuring atomicity) [7, 17, 20]. Thus, the development of efficient synchronization mechanisms that uphold memory consistency and atomicity serves as an essential pillar for constructing high-performance multi-chiplet GPU architectures.

GPUs were originally designed under the assumption that inter-thread synchronization would be coarse-grained and in-

*Corresponding author: Mingyu Wang.

frequent. As a result, they usually adopted a simple, software-driven coherence protocol[16, 18, 25, 38, 41, 47, 49]. As shown in Fig. 1 (a), in traditional GPUs, where L1 cache is private to Streaming Multiprocessors (SMs) and last level cache (LLC) is shared by all SMs which maintains a consistent view of data. These protocols typically invalidate all valid data from the local cache (L1 cache) at acquire operations and flush all dirty data from the local cache at release operations. And to ensure the atomicity and consistency of shared data, synchronization operations (usually atomic operations) were executed at the LLC (i.e., bypassing L1 caches).

However, the changes in multi-chiplet GPU architectures have exacerbated the overhead of such synchronization mechanisms. One of the most notable characteristics of Multi-chiplet GPUs is the Non-Uniform Memory Access (NUMA) problem caused by the bandwidth mismatch between inter-chiplet and intra-chiplet. To mitigate the NUMA problem, multi-chiplet GPUs [2, 8, 29, 52] often incorporate an additional cache level within each chiplet to leverage intra-chiplet data locality and reduce remote accesses. However, this approach results in the globally shared ordering point being located at a lower level, necessitating the invalidation/flushing of the additional cache level during global synchronization. Meanwhile, since synchronization operations are typically executed in the LLC (bypassing other caches), the limited bandwidth of inter-chiplet links still significantly impedes the execution of cross-chiplet synchronization operations. Therefore, as illustrated in Fig. 1(b), the two critical bottlenecks for synchronization are the additional synchronization overhead introduced by the deeper cache hierarchy and the bandwidth limitation of inter-chiplet links for cross-chiplet synchronization operations.

To evaluate the impacts of the additional cache level and the limited inter-chiplet link bandwidth, we compared the workload performance of a 4-chiplet GPU system (as described in the section IV) with that of an equivalent but non-fabricable monolithic GPU. As illustrated in Fig. 2, these workloads involving global synchronization exhibit a significant performance degradation, with an average performance loss of 22.5% due to additional cache-level invalidations and 23.5% due to remote atomic accesses. Overall, the performance of the MCM-GPU decreases by an average of 50.5% compared to the monolithic GPU. These results indicate that efficient data synchronization represents a critical challenge in chiplet-based GPU systems.

Previous research [2, 52, 53] on multi-chiplet GPUs has primarily explored mitigating the NUMA problem arising from bandwidth mismatches between inter-chiplet and intra-chiplet communication for regular data accesses, yet few have addressed the challenges of data synchronization in multi-chiplet GPUs. HMG [43] extends the coherence protocol to multi-chiplet and multi-GPU systems by hierarchically tracking sharers, confining cache invalidation and flushing to the L1 cache level due to the maintenance of the coherence protocol. However, we observe that the complexity of HMG is unnecessary and can sometimes impair performance. CPelide [8] leverages the common processor to track data structures

sent to each chiplet, but it focuses on implicit synchronization to exploit inter-kernel data reuse, rather than optimizing explicit synchronization in GPUs.

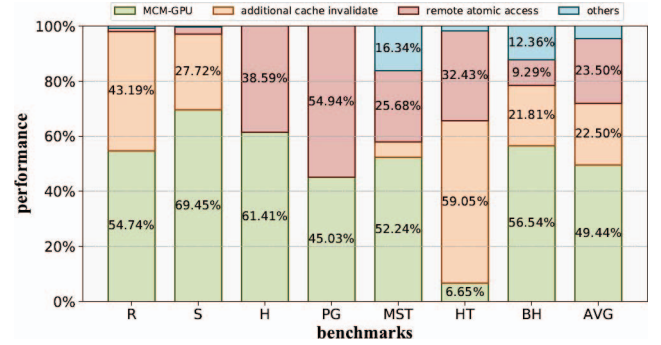


Fig. 2. Performance loss in MCM-GPU[2] versus equivalent monolithic GPU due to synchronization overhead from additional cache level and remote access (experiments are introduced in detail in section IV).

To address these challenges, this paper proposes LRM-GPU to alleviate the synchronization overhead introduced by the additional cache hierarchy and the limited bandwidth of inter-chiplet links. The main insight of LRM-GPU is to leverage the locality among synchronization. Firstly, the locality of synchronization behavior, when a synchronization operation is performed within a chiplet, it is likely that a subsequent synchronization operation will also occur on the same chiplet. Therefore, compared to conservatively invalidating or flushing caches on every synchronization operation, a more positive strategy can be adopted to selectively perform cache invalidation and flushing at appropriate times, thereby improving reuse and reducing synchronization overhead. Secondly, the locality of synchronization data, although atomic operations for synchronization in GPUs are typically bypassed directly to the LLC for execution, cross-chiplet atomic operations issued by different SMs may simultaneously access the same address. Therefore, we can attempt to merge these atomic operations when these operations are sent to the network, thereby reducing inter-chiplet traffic and alleviating the bandwidth pressure between chiplets. In this paper, we make the following contributions:

- 1) To mitigate the synchronization overhead caused by the additional cache level, LRM-GPU proposes an efficient synchronization support by leveraging lazy release consistency in multi-chiplet GPUs. LRM-GPU ensures that coherence actions of the additional cache level are only performed when ownership of synchronization variables is transferred between different chiplets, thereby leveraging the locality of intra-chiplet synchronization and reducing redundant cache invalidations and flushes. And to track the ownership of synchronization variables, LRM-GPU implements a directory in the LLC.

- 2) To mitigate the overhead of inter-chiplet bandwidth limitation encountered when synchronization operations are executed across different chiplets, LRM-GPU proposes in-network atomic merge for synchronization. It embeds a synchronization atomic merging unit within the network. It identi-

fies atomic operations that are being transmitted across chiplets and performs merging processing. By merging these atomic operations at the network, it effectively reduces the traffic of synchronization operations that need to be transmitted across chiplets and alleviates the bandwidth pressure among chiplets.

3) To evaluate the proposed LRM-GPU, we faithfully extended GPGPU-SIM for the multi-chiplet GPU system, including composable network construction and heterogeneous router/crossbar microarchitectures. We have demonstrated the effectiveness of LRM-GPU through a set of detailed experiments conducted on programs with global synchronization. The evaluation results indicate that, compared to MCM-GPU, LRM-GPU achieves an average speedup of $1.33\times$. Moreover, compared to the state-of-the-art work HMG, LRM-GPU also achieves an average speedup of $1.22\times$.

II. BACKGROUND AND MOTIVATION

A. Multi-Chiplet GPU Architecture

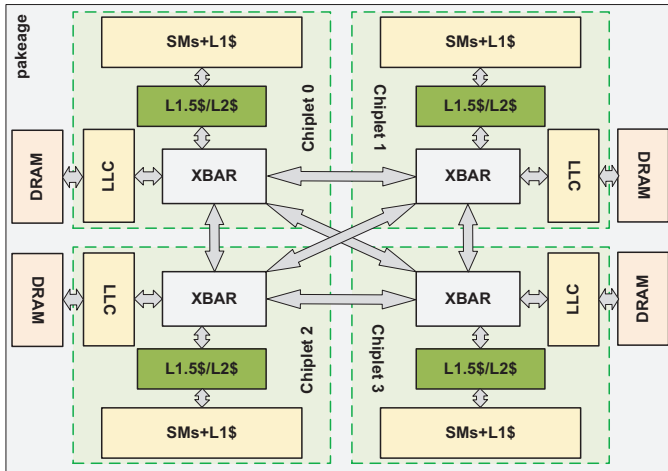


Fig. 3. Multi-Chiplet GPU architecture. Multiple GPU chiplets are used to build a larger logical single GPU.

Fig. 3 illustrates the architecture of a 4-chiplet GPU. Each GPU chiplet contains multiple SMs along with their private L1 cache, as well as a memory-side LLC and a memory partition. All memory partitions offer a globally shared memory address space across all GPU chiplets. The network route memory accesses to the appropriate destination (either the local LLC or the remote LLC) based on the address. The LLC, which is shared by all SMs across the chiplets, exclusively caches data from its local DRAM partition. Consequently, each cache line has only one location, and cache coherence across LLC banks is not required.

Unlike conventional monolithic GPUs, multi-chiplet GPUs often introduce an additional cache level shared within each chiplet to alleviate the NUMA problem caused by bandwidth mismatch between intra-chiplet and inter-chiplet. MCM-GPU [2] uses an L1.5 cache to only cache data from remote chiplets in order to better utilize the data locality within chiplets. In this design, when an L1 miss occurs and data needs to be fetched from the remote chiplet, the request first accesses the L1.5 cache. If the L1.5 cache also misses, the request is routed

to the appropriate LLC and memory partition according to the address mapping policy. The results of MCM-GPU also indicate that the L1.5 cache can achieve good performance. CPELide [8] uses an L2 cache as the shared cache level within a chiplet to exploit the locality of data within chiplets, and the L3 cache (also the LLC) is the global shared cache across all chiplets. Some studies [43, 52] did not add a new cache level, but used LLC (also the L2 cache) on the SM-side, where each chiplet's LLC caches data from all DRAM partitions globally.

Prior research has mainly focused on mitigating the bandwidth constraints between different chiplets due to the limited bandwidth of inter-chiplet links in multi-chiplet GPUs for regular data accesses. As mentioned above, adding an additional cache level or employing SM-side LLCs aims to exploit intra-chiplet data locality to reduce remote accesses. MCM-GPU [2] also proposes the first-touch-page and distributed cooperative thread arrays (CTA) scheduling strategies to increase and leverage data locality within a chiplet. AdCoalescer [53] introduces an adaptive coalescer that uses program counter (PC) values to identify memory requests with high data locality and filter redundant remote reads. Nearfetch [55] targets many-chiplet GPU architectures and proposes fetching data from nearby chiplets to reduce long-distance data transfers.

While these approaches alleviate the general NUMA issues of regular data accesses, they overlook memory access caused by atomic operations for synchronization and the additional overhead brought by synchronization behavior.

B. Synchronization Challenges in Multi-Chiplet GPUs

GPUs were originally designed under the assumption that inter-thread synchronization would be coarse-grained and infrequent. As a result, GPUs often adopt simple, coarse-grained, software-managed coherence protocols. These protocols typically require that private caches be invalidated/flushed upon acquire/release operations, while atomic operations for synchronization are usually performed directly in the LLC.

However, in order to alleviate the NUMA problem in multi-chiplet GPUs, multi-chiplet GPU architecture often adds an additional cache level or uses SM-side LLC to utilize the data locality within the chiplet and reduce remote memory access. These changes in multi-chiplet GPU architectures have exacerbated the synchronization overhead. In particular, whether using an additional cache level or the SM-side LLC, they will all cause the lower global shared ordering point and require maintaining data consistency across multiple levels of the cache hierarchy, leading to more expensive synchronization overhead. For example, when a SM issues an acquire synchronization request, since the L1 and L1.5 caches are private caches within the SM and the chiplet respectively, they may cache stale data. Therefore, it is necessary to invalidate the data in both the L1 and L1.5 cache to ensure that subsequent memory accesses can obtain the latest data from the globally shared LLC. Compared to the monolithic GPU, the multi-chiplet GPU requires the additional invalidation of the L1.5 cache. Given that the capacity of the L1.5 cache is typically larger than that of the L1 cache private to a single SM, and

that the L1.5 cache is shared by all SMs within the chiplet, invalidating the L1.5 cache incurs greater overhead compared to invalidating only the L1 cache and also affects memory accesses from other SMs.

In addition, to ensure atomicity and consistency, atomic operations for synchronization on GPUs are typically executed in the globally shared LLC, and synchronization data is not cached in the L1 and L1.5 caches. This renders the additional cache level ineffective in reducing cross-chiplet atomic operations for synchronization. When it comes to global atomic operations that cross different chiplets, the limitation imposed by inter-chiplet bandwidth becomes a pronounced bottleneck. The need to frequently communicate and synchronize data across chiplets via a potentially constrained bandwidth link can significantly hinder performance, especially in scenarios involving frequent atomic operations for synchronization. Therefore, for global atomic operations for synchronization, the limitation of inter-chiplet bandwidth remains a significant challenge.

C. To Alleviate the Synchronization Overhead

To improve synchronization performance, prior work has actively explored GPU synchronization mechanisms.

DeNovo [47] introduced the DeNovo consistency protocol into GPUs, achieving synchronization by acquiring ownership of the written data. Furthermore, it enhanced DeNovo by adding a selective invalidation mechanism, which can avoid invalidating read-only data regions during synchronization, thereby further optimizing performance. hLRC [1] introduced a mechanism similar to DeNovo to track the ownership of synchronization variables and only performed coherence operations when synchronization variables resided in different L1 caches. However, each time a synchronization operation for the same variable was performed in a different SM, they first needed to invalidate or flush the copies cached in remote L1 caches, to guarantee atomicity and consistency of the synchronization variables. This incurred significant overhead in maintaining atomicity and consistency of synchronization variables across different cache levels. In multi-chiplet GPUs, due to deeper cache hierarchies, this overhead is greatly amplified and can severely degrade performance. Moreover, LAB [10] proposes utilizing a portion of the L1 cache as an atomic buffer to cache atomic data, thereby reducing atomic synchronization traffic sent over the network. Atomic Cache [54] suggests implementing the atomic operations through in-cache computing, thus mitigating synchronization overhead. ARC[11] recommends leveraging the warp-level locality of atomic operations to merge atomic operations in a warp, aiming to alleviate the pressure on the load-store unit and the LLC. However, all of these approaches overlook the inter-SM locality inherent in atomic operations for synchronization.

For multi-chiplet GPUs, HMG[43] extended the cache coherence protocol to multi-chiplet GPUs and multi-GPUs, using a mechanism similar to the VI protocol to track coherence states. By employing an optimized hierarchical sharer-tracking scheme, it alleviated synchronization overhead. HMG lever-

aged the scopes of modern GPU memory models and the non-multiple-copy-atomicity characteristic to avoid the overhead of invalidation acknowledgments and transient states, thereby achieving relatively favorable results. CPElide[8] proposed a new command processor (CP) design, which, by leveraging the global view of the GPU's CP, could selectively decide when to perform implicit inter-kernel synchronization operations. This ensured data was invalidated or flushed before being used, improving data reuse and reducing the overhead of implicit synchronization. However, CPElide lacks efficient support for explicit intra-kernel synchronization.

III. PROPOSED METHODOLOGY

A. Overview

To mitigate the synchronization overhead caused by the additional cache level and the limited bandwidth of inter-chiplet interconnects, this paper proposes LRM-GPU, which provides efficient synchronization support for multi-chiplet GPUs. Fig. 4 presents an architectural overview of the proposed LRM-GPU design.

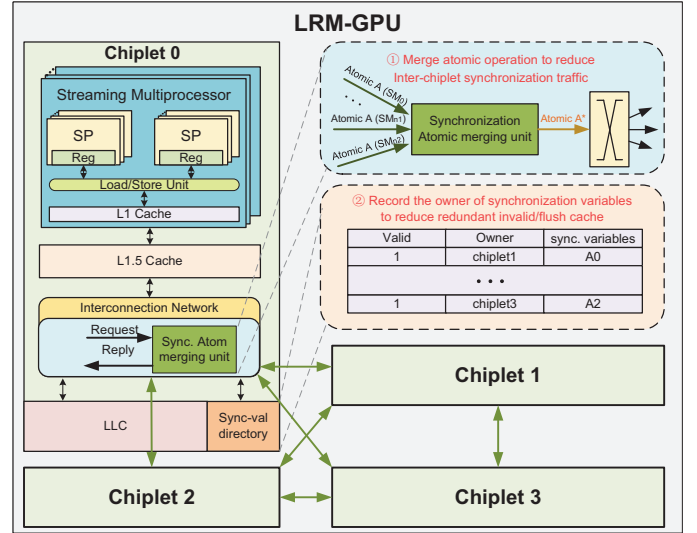


Fig. 4. Overview of LRM-GPU Architecture

LRM-GPU leverages lazy release consistency to mitigate the expensive global synchronization overhead introduced by the additional cache level in multi-chiplet GPU architectures. LRM-GPU associates each synchronization variable with the chiplet that last accessed it. Coherence actions (invalidate/flush cache) are only triggered when the owner of the synchronization variable is transferred between different chiplets, as this indicates the possibility of synchronization between different chiplets. By adopting this approach, LRM-GPU achieves lazy release, thereby leveraging the locality of intra-chiplet synchronization and reducing redundant cache invalidations and flushes. In order to track the ownership of synchronization variables across chiplets, as shown in Fig. 4, LRM-GPU implements a directory at the LLC. Since the directory only monitors synchronization variables rather than all data, its capacity requirements remain modest. LRM-GPU is similar to traditional monolithic GPU designs that recommend all

synchronization operations be bypassed to the LLC. This design ensures that synchronization variables are never cached in the local cache, thereby eliminating the presence of inconsistent replicas and avoiding the costly coherence maintenance for atomicity and consistency across the multi-chiplet cache hierarchy.

The performance of atomic operations for synchronization is inherently limited by the inter-chiplet communication bandwidth since they bypass local caches and are directly routed to the LLC via the network. To alleviate this bottleneck, LRM-GPU introduces in-network atomic merge mechanism and embeds a synchronization atomic merging unit (AMU) within the network, as shown in Fig. 4. AMU identifies and merges atomic requests that traverse chiplet boundaries, thereby reducing inter-chiplet traffic. For example, in lock-based synchronization implemented via atomicCAS, multiple SMs may issue atomicCAS targeting the same data to acquire the lock. Since only one of these requests can successfully acquire the lock while the others fail and spin, these requests can be merged into a single transmission across chiplets. Similarly, in programs using atomicAdd to synchronize updates of shared variables, such as two atomicAdd(x, 1) can be merged into a single atomicAdd(x, 2). By merging such synchronization atomic operations, AMU significantly alleviates inter-chiplet bandwidth pressure.

B. Lazy Release Consistency on Multi-Chiplet GPU

TABLE I
COHERENCE ACTIONS OF LRM-GPU

	status	action
acquire synchronization	invalid	LD data from LLC invalidate L1.5\$
	local chiplet	set owner & LD data from LLC
	remote chiplet	flush remote L1.5\$ (if write-back) set owner & LD data from LLC invalidate L1.5\$
	evicte	flush owner's L1.5\$ (if write-back) set owner & LD data from LLC invalidate L1.5\$
release synchronization	invalid	ST data to LLC invalidate L1.5\$
	local chiplet	set owner & ST data to LLC
	remote chiplet	flush remote L1.5\$ (if write-back) set owner & ST data to LLC invalidate L1.5\$
	evicte	flush owner's L1.5\$ (if write-back) set owner & ST data to LLC invalidate L1.5\$

LRM-GPU leverages lazy release consistency to achieve efficient synchronization. It tracks the owner of synchronization variables through a directory and performs corresponding coherence actions only when the owner changes occur between different chiplets. Table I specifically illustrates how LRM-GPU implements synchronization operations. We focus on the coherence actions related to synchronization in the additional cache level of multi-chiplet GPUs (L1.5 cache), while disregarding the coherence actions of the L1 cache, as it remains consistent with that of traditional GPUs.

When an acquire operation accesses a synchronization variable, it may encounter four scenarios:

(1) **Invalid:** There is no record of the current synchronization variable in the directory, and there are free entries. The directory allocates an entry for this synchronization variable to track its owner. Meanwhile, the acquire operation reads data from the LLC. Finally, it invalidates the local L1.5 cache to ensure that subsequent memory access can read the globally latest data.

(2) **Local chiplet:** The directory has a record of the current synchronization variable, and the recorded owner is the chiplet that issued the synchronization operation request. The acquire operation directly reads data from the LLC without performing any coherence actions. This is because it indicates that there is no inter-chiplet synchronization, and the latest data is cached in the local L1.5 cache.

(3) **Remote chiplet:** The directory has a record of the current synchronization variable, but the recorded owner is not the chiplet that issued the synchronization operation request. If the L1.5 cache adopts a write-back policy, the remote L1.5 cache is first flushed to write the dirty data back to the LLC. Then, the directory updates the owner to the local chiplet. Simultaneously, the acquire operation reads data from the LLC. Finally, it invalidates the local L1.5 cache to ensure that subsequent memory access operations can read the globally latest data.

(4) **Evicted:** There is no record of the current synchronization variable in the directory, and no free entry. The directory evicts an entry according to a policy such as Least Recently Used (LRU). It is necessary to flush the L1.5 cache of the chiplet that owns the evicted entry to ensure that subsequent memory access operations do not read old data (if write-back policy). At the same time, the directory allocates an entry for the current synchronization operation and records its current owner. Then, the acquire operation reads data from the LLC. Finally, it invalidates the local L1.5 cache.

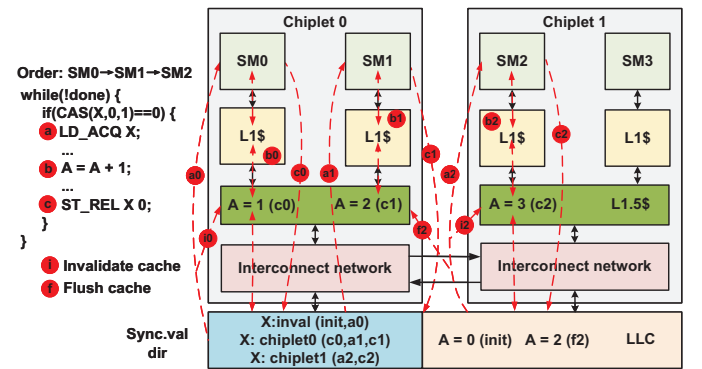


Fig. 5. Example of LRM-GPU synchronization behavior.

The release operation is similar to the acquire operation: (1) Invalid: The directory allocates an entry track for its owner. Simultaneously, the release operation writes the data into the LLC. Finally, it invalidates the local L1.5 cache to ensure that any subsequent acquire operations can achieve correct synchronization. (2) Local chiplet: The release op-

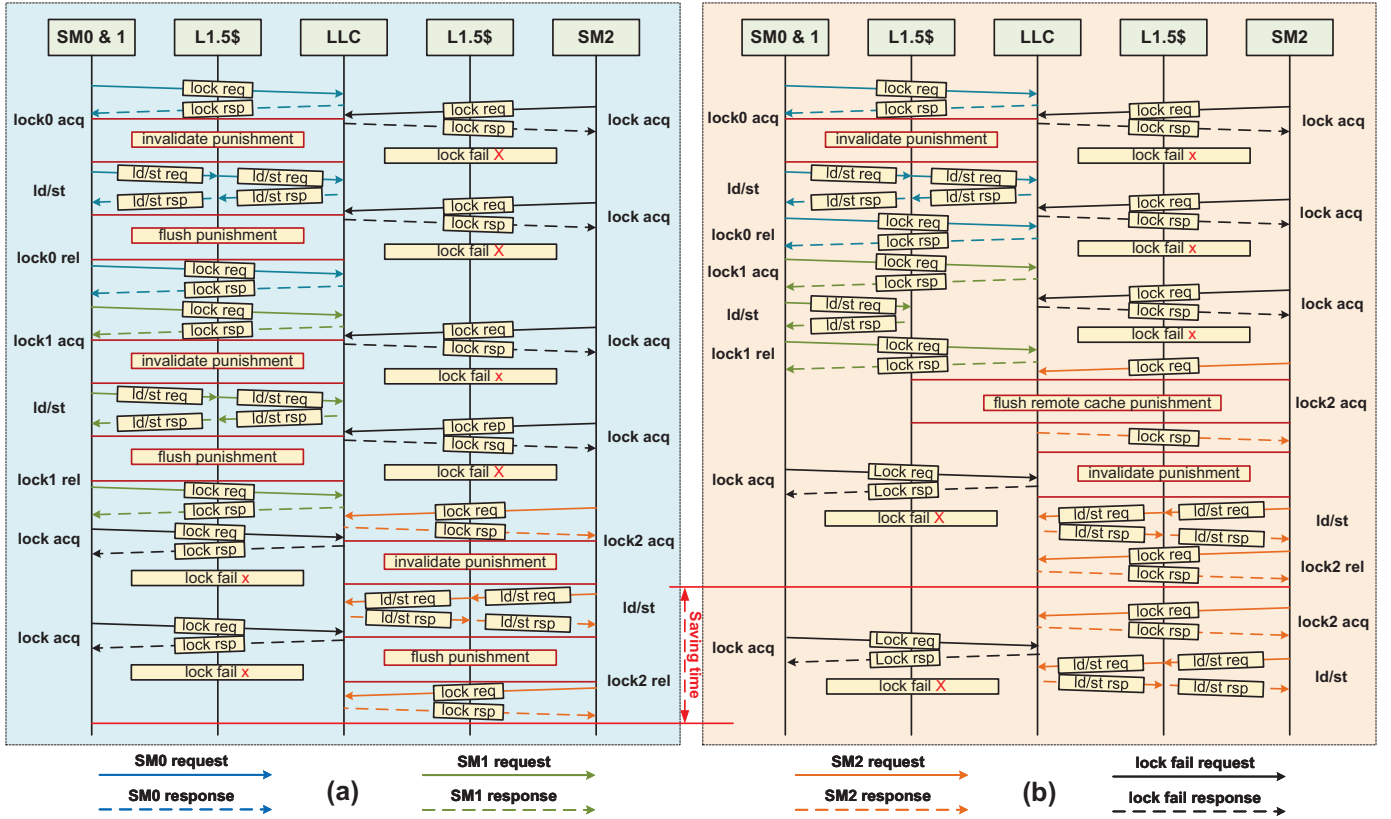


Fig. 6. Comparison of execution flow and implementation between MCM-GPU and LRM-GPU:(a) MCM-GPU; (b) LRM-GPU

eration merely writes the data into the LLC, as coherence actions are delayed until a subsequent change in the owner of the synchronization variable to leverage the locality of intra-chiplet synchronization. (3) Remote chiplet: the remote chiplet's L1.5 cache is first flushed (if write-back policy). Subsequently, the directory updates the owner. Meanwhile, the release operation writes the data into the LLC. Finally, it invalidates the local L1.5 cache. (4) Evicted: it needs to flush the L1.5 cache of the chiplet that owns the evicted entry (if write-back policy). At the same time, the directory allocates an entry for the current synchronization variable and records its current owner. And the release operation write data to the LLC. Finally, it invalidates the local L1.5 cache.

For better understanding, Fig. 5 presents an example of the synchronization execution flow in a lock-based synchronization program. In this example, threads from three SMs are competing for lock synchronization, where SM0 and SM1 are on chiplet0, and SM2 is on chiplet1. Their execution order is SM0 → SM1 → SM2. Here, we also ignore discussions on L1 cache coherence actions and focus on L1.5 cache coherence actions, assuming that the L1.5 cache employs a write-back policy since it is more complex than the write-through policy.

Initially, SM0 successfully acquires the lock (where the atomicCAS operation is directly executed in the LLC). It then issues an acquire synchronization to read the synchronization variable X from the LLC (a0). Since there is no record of synchronization variable X in the directory, the L1.5 cache needs to be invalidated to ensure that SM0 can read the

latest data (i0). Additionally, the owner of X in the directory is set to chiplet0. Subsequently, SM0 performs load/store operations on data A (b0). At this point, the L1.5 cache of chiplet0 holds the latest data for A, with A=1, while the LLC contains the stale data, A=0. Finally, SM0 executes the release operation, directly writing the data of X into the LLC (c0). Since there is a record of X in the directory and its owner is chiplet0, no coherence actions are required. Coherence actions will be delayed until the owner of the synchronization variable X changes. Next, SM1 acquires the lock and issues an acquire synchronization request (a1). Since the owner of X is chiplet0, it indicates that the latest data resides in chiplet0, and there is no need to invalidate the L1.5 cache. SM1 then performs load/store operations on data A (b1), with chiplet0 holding the latest data, A=2. Finally, SM1 executes a release operation (c0), again without performing any coherence actions. Subsequently, SM2 on chiplet1 acquires the lock and issues an acquire synchronization request (a2). Since the recorded owner of the synchronization variable X in the directory is chiplet0 at this time, the L1.5 cache of chiplet0 needs to be flushed to write back the dirty data (f2), enabling chiplet1 to obtain the latest data, A=2, from the LLC. The directory then changes the owner of synchronization variable X to chiplet1 and invalidates the L1.5 cache of chiplet1 to ensure that it can read the latest data from the LLC (i2). SM2 performs load/store operations on data A (b3), obtaining the latest data, A=3. Finally, SM2 executes a release operation without performing any coherence actions (c2).

Fig. 6 provides a detailed illustration of the synchronization execution of the aforementioned program in LRM-GPU, along with a comparison of the synchronization execution flows and implementations between the MCM-GPU and LRM-GPU. As shown in Fig. 6 (a), in the MCM-GPU, each acquire and release synchronization operation necessitates invalidating or flushing the local L1.5 cache, thereby resulting in substantial performance degradation. In contrast, as shown in Fig. 6 (b), LRM-GPU leverages lazy release consistency by tracking the owners of synchronization variables to exploit locality between synchronization operations. It only performs invalidation/flushing of the L1.5 cache when the owner of a synchronization variable changes across different chiplets, thereby reducing redundant synchronization overhead.

C. In-Network Atomic Merging for Synchronization

The Synchronization Atomic Merge Unit (AMU) is designed to merge atomic requests targeting the same address across chiplets in the network, thereby reducing the inter-chiplet synchronization traffic, and each chiplet contains an AMU. Fig. 7 illustrates the detailed structure of AMU. AMU is embedded into the network and primarily comprises four components: the merge table, the instruction decoder, the ALU (Arithmetic Logic Unit), and the multicast unit.

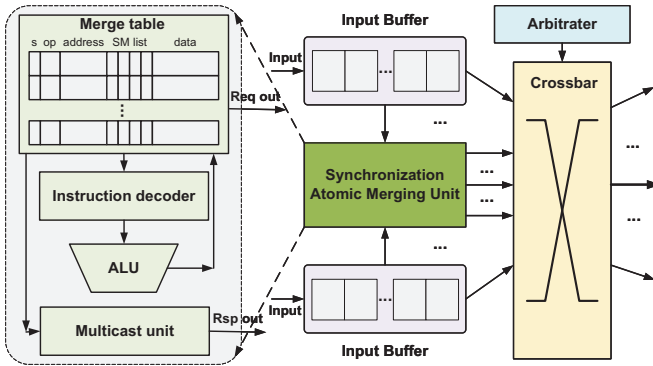


Fig. 7. Synchronization Atomic Merge Unit microarchitecture.

The merge table contains multiple entries, with each entry consisting of five fields: (1) **Status**: determining whether the entry is currently valid. There are three states: "valid," indicating that the entry is valid and new atomic requests can be merged; "reserve," indicating that the entry is valid but new atomic requests cannot be merged because the recorded request has been sent and is awaiting for response; and "invalid," indicating that the entry is invalid. (2) **Opcode**: representing the atomic operation and corresponding operation masks and auxiliary identification information. (3) **Address**: identifying the address that requires access. (4) **SM list**: recording the IDs of SM that have issued atomic requests to the same address. (5) **Data**: recording the data used for the atomic operation to be performed. For instance, an atomic operation issued by SM0 to add 1 at address 0xA0AC can be represented as the tuple (valid, atomadd, 0xA0AC, 0, 1). The instruction decoder is responsible for identifying atomic operations to enable proper merging of synchronous atomic requests. The ALU

performs the correct merging of synchronous atomic requests based on the decoding results. The multicast unit, during reply transmission, broadcasts responses to the corresponding SMs according to the SM IDs recorded in the merge table entries.

To meet the high-concurrency requirements of GPUs, the AMU is designed with multiple input, output, and computational channels. Additionally, the merge table is also devised with a multi-bank structure to enable parallel access. To facilitate rapid querying, we propose separating the storage of the search key (i.e., status, opcode, address) and the data (i.e., SM list, data). Specifically, a Content-Addressable Memory (CAM) is employed for the storage and querying of the search key. The CAM can determine whether there is a matching entry within the CAM and obtain the positional index in a clock cycle. Meanwhile, the data is stored using SRAM. Furthermore, to support simultaneous read and write requests on the merge table, both the CAM and the SRAM adopt a dual-port design.

Next, we discuss the workflow of AMU. It is first checked whether the request is a cross-chiplet request when an atomic request generated by an SM is sent into the network. If it is, the atomic request is forwarded to AMU. Otherwise, the request is sent directly. After entering AMU, the request is initially sent to the merge table. If a corresponding valid entry is found, the atomic request is processed and merged with the data in the table. Conversely, if no entry matches the atomic request's address or there is a match but the status is "reserve", and if there is an available entry, AMU will allocate a new entry for it. Otherwise, the atomic request is sent out directly without being recorded in the table. When allocating a new entry, AMU controller also assigns a countdown timer for it. During this countdown period, the entry can continue to receive and merge new atomic requests. Once the countdown expires, the request in the entry is sent out, and the status is changed from "valid" to "reserve," preventing further request merging. Alternatively, if the merged requests have reached the maximum number (the SM list is full), the request in the entry is also sent out, and the status is changed to "reserve". Upon receiving the response, AMU queries the merge table. If a matching entry is found, AMU broadcasts the response of the atomic operation to these SMs via the multicast unit according to the SM list, and simultaneously resets the entry to make it available for future requests. Conversely, if no entry reflects the response address, AMU directly forwards the response to the designated SM.

Fig. 8 presents an example of AMU's workflow. Initially, the merge table contains an entry for the atomicadd(addr0, 1) request issued by SM1 ①. Subsequently, SM0 and SM1 issue cross-chiplet atomicadd(addr0, 1) ② and atomicadd(addr1, 1) ③ requests, respectively. These requests are input into AMU and first query the merge table when they arrive at the network. The atomicadd(addr0, 1) request issued by SM0 finds a matching entry, enabling their merging into atomicadd(addr0, 2), with SM0's ID being recorded. In contrast, the atomicadd(addr1, 1) request issued by SM1 does not find a matching entry, prompting the merge table to allocate a new

entry to record it. After a certain period, SM0 and SM1 issue cross-chiplet `atomicadd(addr1, 2)` ④ and `atomicadd(addr0, 3)` ⑥ requests again, respectively. At this point, the request previously recorded as `atomicadd(addr0, 2)` in the merge table has already been dispatched from AMU ⑤, and its status has been updated to "reserve," indicating that it is awaiting a response. Consequently, the `atomicadd(addr0, 3)` request issued by SM1 cannot be merged with this entry and must either be allocated a new entry or be sent out directly. Meanwhile, the `atomicadd(addr1, 2)` request issued by SM0 finds a matching entry, allowing it to be merged with the data in the entry to produce `atomicadd(addr1, 3)`. The responses also enter AMU and query the merge table when the responses to these requests are received. If a matching entry exists, AMU broadcasts the reply to the SMs within the chiplet based on the IDs recorded in the SM list of the entry.

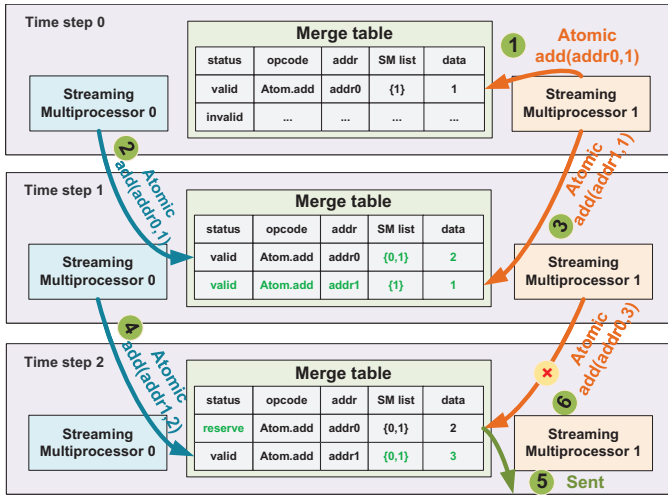


Fig. 8. Example of AMU's workflow.

Although the aforementioned discussion has focused solely on examples of the `atomicadd` operation, the principles and methodologies apply similarly to other atomic operations. Table II lists the atomic operations supported by the AMU. When the AMU detects these free atoms (commutative/unordered atomics), it can attempt to combine multiple atomic requests targeting the same address into a single aggregated request by performing the corresponding computations. Among them, `atomiccas` is a special case: for operations on the same address, it additionally requires that the comparison data of the two requests be identical. When the comparison data are the same, one of the requests is selected as the combined request and sent to the LLC to attempt the data exchange, while the other requests are guaranteed to fail and will wait for the response of the combined request before being returned to their corresponding SMs.

Moreover, in reality, memory accesses and data transfers in practice often operate at coarser address granularity, such as cache line granularity. Accordingly, atomic requests falling within the same coarse-grained address region can be merged to improve bandwidth utilization. For requests targeting different offsets within the same region, the merging is performed

TABLE II
EVALUATED BENCHMARKS

Operation	Description
<code>atomicand</code>	combined by request1's val & request2's val.
<code>atomicor</code>	combined by request1's val request2's val.
<code>atomicxor</code>	combined by request1's val $\oplus$ request2's val.
<code>atomicadd</code>	combined by request1's val + request2's val.
<code>atomicsub</code>	combined by request1's val - request2's val.
<code>atomicmax</code>	compares request1's val and request2's val, updating with the maximum value
<code>atomicmin</code>	compares request1's val and request2's val, updating with the minimum value
<code>atomiccas</code>	only when the comparison data is same can they be combined, and at most one can be successfully swaps while the others fail

using an operation-mask-based approach that merges multiple atomic operations into a single transaction.

IV. EXPERIMENTAL SETUP

A. System Setup

In the evaluation, we faithfully extended GPGPU-Sim [24] to simulate the multi-chiplet GPU system and integrated the proposed LRM-GPU into the simulator. In particular, we have developed a chiplet interconnection platform based on Book-Sim 2.0 [19], which encompasses features such as composable network construction and heterogeneous router/crossbar microarchitecture. In this platform, the network within a chiplet is independently designed and configured according to its original NoC architectures. During the chiplet integration phase, the platform takes these pre-designed intra-chiplet networks as inputs and constructs a composable inter-chiplet interconnect through a process involving network integration, modular routing, and chiplet interface incorporation. This platform has been integrated into the GPGPU-Sim simulator, enabling GPGPU-Sim to accurately model and uniformly evaluate the entire multi-chiplet system.

As shown in Table III, the evaluated multi-chiplet GPU system comprises 4 chiplets, with each chiplet containing 64 SMs, resulting in a total of 256 SMs. Each SM is equipped with a private L1 cache with a capacity of 128 KB, employing a write-through policy. Simultaneously, each chiplet features an intra-chiplet shared L1.5 cache that exclusively caches remote data, also utilizing a write-through policy. The LLC is globally shared across all chiplets and adopts a write-back policy. The total capacity of the L1.5 and LLC caches is 16 MB. All NoCs examined in this paper employ concentrated hierarchical crossbar topologies, with the inter-chiplet network directly connecting each GPU chiplet to every other GPU chiplet, as shown in Fig. 4. The total inter-chiplet bandwidth is 768 GB/s, and each hop in the inter-chiplet network incurs a latency of 32 cycles. We assume the presence of 4 memory partitions, culminating in a total of 64 memory channels and a memory bandwidth of 3 TB/s. We adopt a first-touch page allocation policy [2], which maps pages to the memory partition of the GPU chiplet that first accesses data within the

page. Furthermore, we consider distributed CTA scheduling [2], which partitions the set of CTAs by assigning each chiplet a contiguous group of CTAs to maximize data locality among CTAs within the same chiplet. These configurations are consistent with previous works [2, 52, 53]. For LRM-GPU, the synchronization variable directory comprises a total of 64 entries. And each chiplet is equipped with an AMU, which features 16 channels. Its merge table contains 2K entries, organized into 16 banks, and each entry maintains the SM list that can accommodate up to 8 SM IDs.

TABLE III
MULTI-CHIPLET GPU SYSTEM CONFIGURATIONS

Number of chiplets	4
Total number of SMs.	256
Max number of warps	64 warps/SM, 32 threads/warp
L1 data cache	128 KB/SM, 128B lines, 4 ways, write through
L1.5 cache	2 MB/chiplet, 128B lines, 16 ways, write through
LLC	8 MB, 128B lines, 16 ways, write back
Total capacity of L1.5 cache and LLC cache	16 MB
Inter-chiplet interconnect	768GB/s, 32 cycles/hop
Total DRAM bandwidth	64 channels, 3 TB/s
Page size/allocation	4KB, first-touch
CTA allocation	Distributed CTA scheduling
Sync. variable directory	64 entries, 16 entries/chiplet
AMU	16 channels; 2k entries, 16 banks; SM list: 8; data: 32B

B. Configurations

To determine the effectiveness of LRM-GPU, we evaluated the following configurations:

MCM-GPU [2]: MCM-GPU employs techniques such as L1.5 cache, first-touch page allocation, and distributed CTA scheduling to enhance the performance of multi-chiplet GPUs. Its synchronization mechanism is similar to that of traditional GPUs. During an acquire operation, the L1 and L1.5 caches are invalidated. It is necessary to ensure that dirty data is flushed back to the LLC when a release operation occurs. Since both the L1 and L1.5 caches adopt a write-through policy, stale data is guaranteed not to exist in the LLC, thus eliminating the need to flush the L1 and L1.5 caches. This principle also applies to other configurations.

hLRC [1]: It was originally designed for monolithic GPUs, we implemented it in GPGPU-Sim and extended its described mechanisms to multi-chiplet GPUs. hLRC tracks the registration locations of synchronization variables to exploit synchronization locality among SMs. It also caches synchronization variables in caches at all levels and employs a write-back policy for them. To ensure the consistency and atomicity of synchronization variables, whenever the location of a synchronization variable changes, the remotely cached synchroniza-

tion variable must be written back. Until this write-back is completed, no other thread can access this synchronization variable. For relaxed atomic operations, the traditional GPU approach is still adopted.

HMG (NHCC)[43]: HMG is a state-of-the-art chiplet-based, multi-GPU coherence protocol. Although primarily designed for multi-GPU systems, we focused on its application in multi-chiplet GPUs and implemented it in GPGPU-Sim. HMG utilizes an SM-side LLC instead of an L1.5 cache. The LLC uses the write-through policy, which writes all data to the home node of the LLC and memory partition. Therefore, the home node always contains the latest data. Consequently, HMG employs an LLC coherence directory to track all data and maintain the LLC coherence, with each GPU chiplet having 12K entries, each entry covering four cache lines.

TABLE IV
EVALUATED BENCHMARKS

Microbenchmarks [48] (inputs: 8 512 2)			
atomicTreeBarr (ATB)	lfTreeBarr (LTB)	spinMutex (SPM)	sleepMutex (SLM)
faMutex (FAM)	spinSem2 (SPS2)	spinSem10 (SPS10)	spinSem120 (SPS120)
with global synchronization			
benchmarks	inputs	benchmarks	inputs
reduce(R)[9]	8192 32	scan(S)[9]	16384
histogram(H)[28]	262144 256	pagerank(P)[4]	coAuthorsDBLP .graph
barnes-hut (BH)[31]	262144 4 0	hash-table (HT)[51]	65536 2048
minimum spanning tree(MST)[31]		USA-road-d.NY.gr	
without global synchronization			
benchmarks	inputs	benchmarks	inputs
b+tree(BT)[5]	command.txt	backprop(BP)[5]	65536
bfs[5]	graph65536.txt	dwt2d[5]	192.bmp
nn[5]	filelist_32	lavaMD[5]	10×10×10
vgg16_fw(vgg)[33]	–	gpt2_fw(gpt)[32]	–
hotspot(HS)[5]	temp_512, power_512		

C. Workloads

We focus on applications that necessitate global synchronization. Moreover, to evaluate whether the proposed design would have any adverse impact on other applications, we also assessed applications that do not involve global synchronization. Table IV provides a summary of these workloads, which have also been widely used in other studies[12, 13, 54] to investigate GPU synchronization issues. For these applications, we configured their input sizes to ensure reasonable occupancy on the multi-chiplet GPU. Additionally, to thoroughly test our proposed scheme, we made minor modifications to the workloads to incorporate synchronization semantics such as acquire and release. The multi-chiplet GPU architecture preserves the monolithic GPU abstraction, all chiplets present a unified global memory space and a single logical device to the programming model. Consequently, the workloads' kernels, data structures, and launch configurations do not need to be

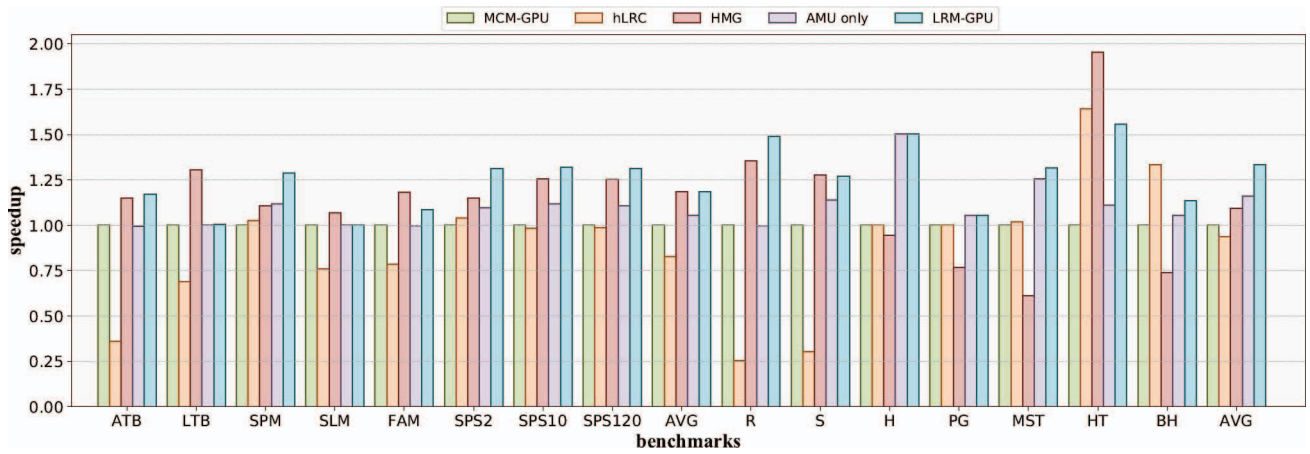


Fig. 9. Speedup on MCM-GPU, hLRC, HMG, AMU only and LRM-GPU.

modified; the only differences lie in how the hardware and runtime partition data and schedule threads across chiplets, which is completely transparent to the programmer. These workloads were compiled using the O3 optimization level under CUDA 11.1. In the evaluation, we focus on the kernels that need to be global synchronized for these workloads.

V. EXPERIMENTAL RESULTS

A. Performance Evaluation

In order to quantify the performance advantages of LRM-GPU, we evaluated multiple workloads with global synchronization, as well as microbenchmarks with different synchronization schemes, using MCM-GPU, hLRC, HMG, AMU only and LRM-GPU. The speedups are illustrated in Fig. 9, with all speedups normalized to MCM-GPU as the baseline. The simulation results show that for microbenchmarks with various synchronization schemes (including barriers, lock-based synchronization, and semaphores) LRM-GPU achieves good speedup, with an average performance improvement of $1.19\times$, demonstrating its effectiveness and adaptability. For most benchmarks requiring global synchronization, LRM-GPU also achieves the highest speedup, indicating that it effectively mitigates synchronization overhead in multi-chiplet GPUs. Specifically, compared to MCM-GPU, LRM-GPU attains an average speedup of $1.33\times$, with the AMU contributing an average speedup of $1.16\times$. Compared to the state-of-the-art work HMG, which extends cache coherence in multi-chiplet GPUs, LRM-GPU achieves an average speedup of $1.22\times$. hLRC performs poorly in most benchmarks because it caches synchronization variables in multiple levels of caches. Although this approach better exploits the locality among synchronization variables, each time a synchronization variable is accessed by different SMs, it initially misses in the local caches at all levels and necessitates flushing the synchronization variable cached in the remote L1 cache back to the LLC. During this period, all other threads are unable to obtain its registration and access it. Consequently, when contention for synchronization variables is intense, the frequent cache misses across multiple cache levels and the waiting for synchronization variable flush

significantly degrade its performance. HMG achieves good performance on some benchmarks, even attaining the highest speedup in HT. However, it also performs poorly on some benchmarks, such as MST and PG, because these benchmarks contain a large number of atomic operations, which trigger numerous write-invalidations in HMG, leading to degraded performance.

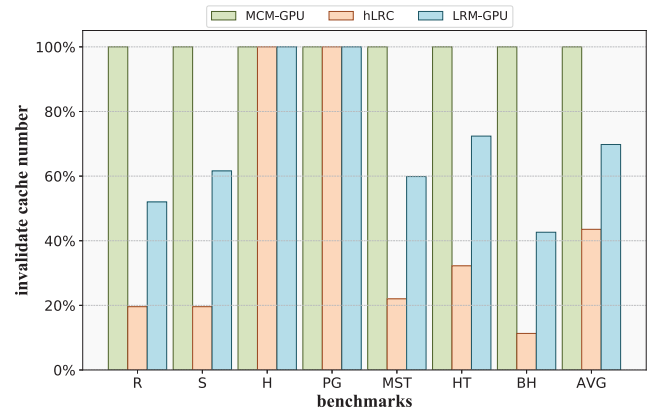


Fig. 10. Invalidate cache number on MCM-GPU, hLRC, and LRM-GPU.

Fig. 10 illustrates a comparison of the number of invalidating L1.5 cache among MCM-GPU, hLRC, and LRM-GPU, with all values normalized to MCM-GPU as the baseline. Since HMG employs a cache coherence protocol to maintain data coherence in the SM-side LLC, it does not require cache invalidations or flushes beyond the L1 cache; hence, it is excluded from this comparison. Compared with MCM-GPU, LRM-GPU reduces the number of L1.5 cache invalidations by an average of 30%, while hLRC achieves a 56% reduction. For hLRC, since it caches synchronization variables in both L1 cache and L1.5 cache, local synchronization requests can access these variables more rapidly and are more easily able to acquire the registration for them compared to remote synchronization requests, thereby better leveraging locality. Consequently, in most scenarios, hLRC experiences fewer occurrences of invalid cache accesses than LRM-GPU. However, as previously mentioned, when synchronization requests are

issued by different SMs, synchronization variables cached remotely need to be written back. During the write-back period, other synchronization requests attempting to access the same variable will fail, significantly increasing synchronization latency. This undermines the advantage of reducing the number of invalid cache accesses, preventing it from being effectively exploited. Regarding the histogram and pagerank benchmarks, which primarily utilize atomic operations for synchronous updates of shared data, invalidating the cache occurs only upon kernel completion. Therefore, there is virtually no reduction in the number of invalid cache accesses for both hLRC and LRM-GPU in these scenarios.

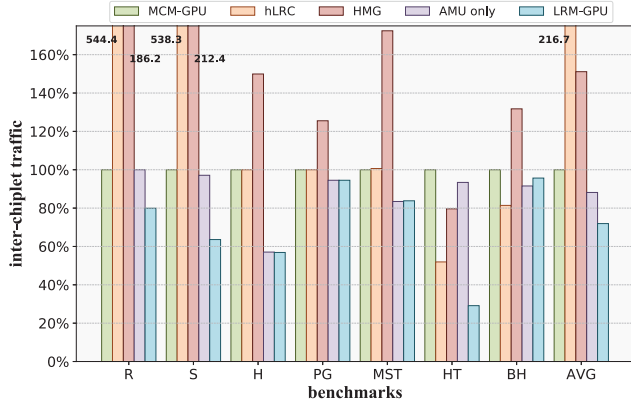


Fig. 11. Inter-chiplet traffic on MCM-GPU, hLRC, HMG, AMU only and LRM-GPU.

Fig. 11 presents a comparison of inter-chiplet traffic among MCM-GPU, hLRC, HMG, AMU only and LRM-GPU. All traffic values normalized to MCM-GPU as the baseline, and the traffic here includes all memory accesses and coherence messages. Compared with MCM-GPU, LRM-GPU reduces inter-chiplet traffic by 28%, with the AMU contributing a 12% reduction in traffic. Moreover, compared with the state-of-the-art HMG design, it achieves an average reduction of 52%. For hLRC, since synchronization variables are cached across various cache levels, each time a synchronization variable is accessed by different SMs, a write-back request must be sent to the remote cache holding the synchronization variable. And, other threads cannot successfully register the synchronization variable until the write-back operation is completed. To prevent blocking memory accesses, each failed synchronization access triggers a return and resending of the request. Consequently, when contention for synchronization variables is intense, inter-chiplet traffic significantly increases. This explains the reason that hLRC still suffers performance degradation despite fewer cache invalidations. HMG employs the cache coherence protocol, necessitating the transmission of an invalidation request to sharers of written data. As a result, in most scenarios, HMG exhibits increased inter-chiplet traffic. However, since HMG does not require invalidation of caches beyond the L1 level and does not need to receive an invalidation reply, it still achieves a positive performance impact overall. In contrast, LRM-GPU effectively reduces inter-chiplet traffic in most cases and decreases the number

of cache invalidations, thereby achieving a notable speedup.

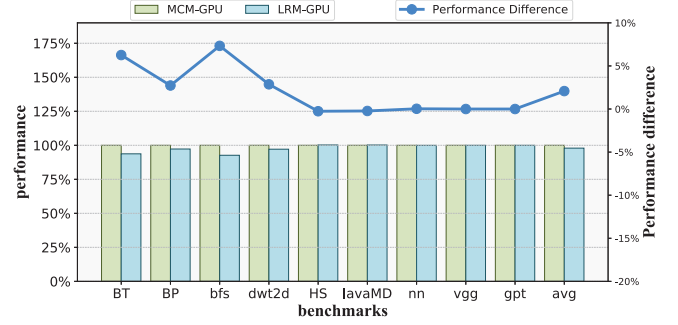


Fig. 12. Performance of benchmarks without global synchronization on MCM-GPU and LRM-GPU.

To assess the impact of the LRM-GPU modifications for multi-chiplet GPUs on the execution of other programs, we tested a set of benchmarks that do not involve global synchronization, including ML inference programs such as VGG16 and GPT-2. The results are shown in Fig. 12, with all performance normalized to MCM-GPU as the baseline. The results indicate that the modifications implemented in LRM-GPU have minimal impact on the execution of other programs, with an average performance difference of only 2% compared to MCM-GPU.

B. Energy and Area Evaluation

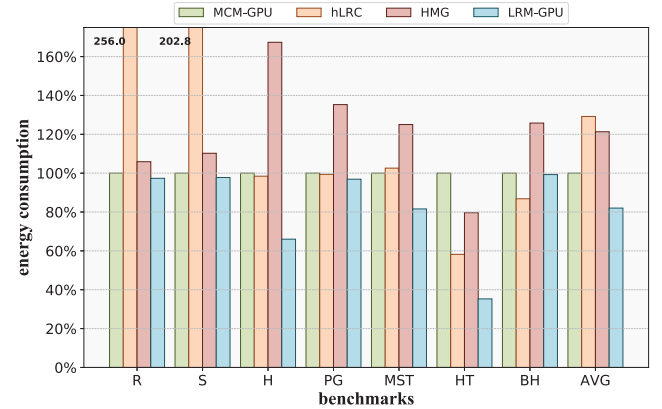


Fig. 13. Energy consumption on MCM-GPU, hLRC, HMG, and LRM-GPU.

To evaluate the energy consumption of LRM-GPU system, we analyzed runtime activities to calculate dynamic power. We leveraged AccelWatch [21] of GPGPU-Sim and inter-chiplet transmission energy data (0.54 pJ/bit) provided in the paper [2, 40] to evaluate the energy consumption of the cache system and Network under different benchmarks. Fig. 13 presents a comparison of energy consumption among MCM-GPU, hLRC, HMG, and LRM-GPU, with all energy consumption values normalized to MCM-GPU as the baseline. The results indicate that, compared to MCM-GPU, LRM-GPU achieves an average energy reduction of 18%. Moreover, compared to HMG, LRM-GPU achieves an average energy reduction of 32%. hLRC and HMG exhibit increased energy consumption due to the increased inter-chiplet transmission traffic.

Fig. 14 presents the energy breakdown of MCM-GPU and LRM-GPU, where the inter-chiplet and intra-chiplet networks account for the majority of the energy consumption. Since LRM-GPU reduces inter-chiplet traffic, the energy share of its inter-chiplet network is slightly lower compared to MCM-GPU, and the newly added AMU consumes only 0.13%.

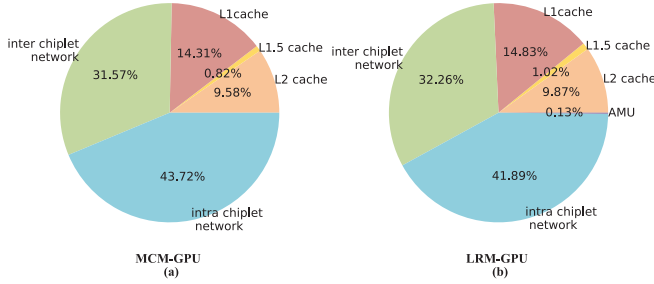


Fig. 14. Energy breakdown of (a) MCM-GPU and (b) LRM-GPU.

To evaluate the proposed AMU that has been embedded into the network, we employed Cadence Virtuoso to customize and implement the merge table within AMU, while the other components of AMU were realized using Verilog RTL. For the merge table, a corresponding netlist was generated under the TSMC 40nm process. This netlist was subsequently integrated into the Cadence Spectre simulation environment and simulated under the TT process corner, at a temperature of 25 °C, a frequency of 1 GHz, and a voltage of 1.1V to measure its power consumption. We then evaluate the area of the merge table based on the layout of the customized circuit and the 40nm standard cell library. For the remaining components of AMU, we utilized Synopsys Design Compiler to synthesize the logic circuits under the TSMC 40nm process. The breakdown of AMU's energy consumption and area utilization is detailed in Table V.

In particular, the total power consumption of AMU is 301mW at 40nm process. To better contextualize this cost, we referred to NVIDIA GPU V100, which comprises 80 SMs, a number comparable to the 64 SMs in a chiplet of our system, is fabricated using a 12nm process with a total power consumption of 300W [37]. Despite the V100 being fabricated in a more advanced process, the power consumption of AMU is only 0.1% of V100. And the total area of AMU is 1.84mm² at 40nm process. For reference, the area of V100 is 815mm² at 12nm process, while the AMU, evaluated in an older technology node, is only 0.2% of the V100's area.

TABLE V
ENERGY AND AREA BREAKDOWN OF AMU

Components	Power(mW)	Area(mm2)
merge table	185.51	1.52
others	115.93	0.32
total	301.44	1.84

In addition, we employ a directory to track the owner of synchronization variables. For the system with 4 chiplets, a 2-bit vector is required to represent the owner. We assume that

the tag address is 48 bits, and an additional 1 bit is needed to indicate whether an entry is valid. Consequently, each entry in the directory needs 51 bits. There are 64 entries in the directory, so the total capacity of the directory is 0.4 KB, which accounts for only 0.3% of the capacity of a L1 cache.

C. Sensitivity analyses

We conducted a sensitivity study and analysis of LRM-GPU under different chiplet scales and inter-chiplet transmission latencies.

Chiplet scale: We increased the number of chiplets from 4 to 6 and then to 8, while keeping the resources on each chiplet unchanged, as shown in Fig. 15 (a). As the number of chiplets increases, the performance gain of LRM-GPU gradually diminishes, dropping from a 1.33× improvement with 4 chiplets to a 1.21× improvement with 8 chiplets. This is expected, since increasing the number of chiplets reduces the locality of synchronization variables across chiplets, which in turn leads to decreased performance.

Inter-chiplet transmission latency: We tested latencies ranging from 8 cycles to 48 cycles, as shown in Fig. 15 (b). The performance of LRM-GPU remains nearly unchanged between 8 and 32 cycles, with only a slight drop at 48 cycles. Due to the high parallelism of GPUs and warp switching that hides latency, GPU performance is relatively insensitive to latency but sensitive to bandwidth.

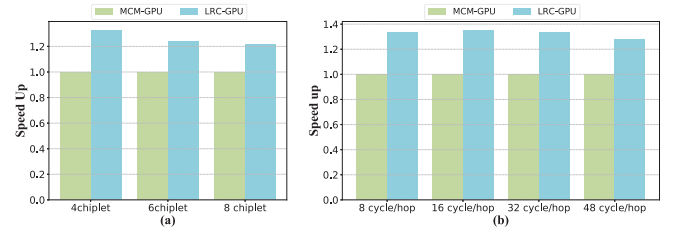


Fig. 15. Sensitivity analyses for LRM-GPU. (a) Chiplet scale; (b) Inter-chiplet transmission latency.

VI. RELATED WORK

Multi-chiplet GPUs. Existing work on multi-chiplet GPUs mainly focuses on NUMA effects and bandwidth mismatch. MCM-GPU[2] reduces remote accesses via remote data caching, distributed CTA scheduling, and first-touch page placement. Memory-centric MCM-GPU[44] builds a bridge network on the interposer to dynamically allocate the GPU's off-chip bandwidth and better match application demands. AdCoalescer[53] and NearFetch[55] reduce cross-chiplet traffic and remote bandwidth usage by coalescing memory requests across SMs and forwarding shared data to nearby requesters along the path, respectively. SAC[52] further adapts to different sharing patterns by dynamically reconfiguring the LLC organization between memory-side and SM-side modes according to cross-chiplet sharing characteristics, thereby improving effective bandwidth. In addition, MGvm[42] proposes an MCM-aware virtual memory design to mitigate the impact of non-uniformity on TLB hit rates and page table walks, while Barre Chord[14] exploits the structural property of

physically isomorphic pages across chiplets to merge multiple translations, reducing both local/remote page table walks and IOMMU access overhead.

GPU synchronization. hLRC[1] introduced a mechanism similar to DeNovo to track the ownership of synchronization variables and only performed coherence operations when synchronization variables resided in different L1 caches. However, in the multi-chiplet GPU, frequent ownership migration across chiplets incurs substantial coherence overhead under limited inter-chiplet bandwidth and deeper cache hierarchies. ARC[11], LAB[10], and DAB[7] merge atomic operations within an SM via warp-level reduction and on-SM atomic buffers, but do not exploit locality across SMs. Atomic Cache[54] implements the atomic operations through in-cache computing, thus mitigating synchronization overhead. HMG[43] extends cache coherence to multi-GPU systems using a VI-like protocol and a hierarchical sharer-tracking scheme to reduce synchronization overhead. CPElide[8] leverages the CP's global information to optimize implicit synchronization, but is ineffective for explicit synchronization because runtime contention is hard to predict. In contrast, LRM-GPU focuses on explicit synchronization in multi-chiplet GPUs, it maintains a lightweight directory of synchronization variables at the LLC and only triggers coherence actions when ownership moves across chiplets; at the same time, AMU embedded in the interconnect network exploit chiplet-level locality to merge cross-chiplet atomic requests, thereby alleviating the limited inter-chiplet bandwidth bottleneck.

VII. CONCLUSION

To alleviate the synchronization overhead in multi-chiplet GPUs caused by additional cache levels and limited inter-chiplet link bandwidth, this paper proposes LRM-GPU to provide efficient synchronization support. LRM-GPU leverages lazy release consistency to mitigate the more expensive global synchronization overhead stemming from the extra cache hierarchy. It associates each synchronization variable with the chiplet where it was last accessed, and only performs coherence actions such as invalidation and flushing when the owner of the synchronization variable changes across different chiplets, thereby reducing redundant cache invalidations and flushes. To address the issue of limited inter-chiplet bandwidth affecting atomic operations for synchronization executed across different chiplets, LRM-GPU introduces a synchronization atomic merge unit within the network. This unit detects atomic operations transmitted across chiplets and merges them to reduce inter-chiplet traffic. The evaluations demonstrate that, compared to the MCM-GPU, LRM-GPU achieves a speedup of $1.33\times$. Moreover, compared to the state-of-the-art HMG, LRM-GPU also achieves the speedup of $1.22\times$, reduces 52% of inter-chiplet traffic, and reduces 32% of energy consumption on average. Additionally, LRM-GPU achieves significant advantages while generating minimal additional overhead. These advantages indicate the significant potential of LRM-GPU, and this solution can be integrated into

emerging multi-chiplet GPU architectures to facilitate efficient synchronization.

ACKNOWLEDGMENT

This work was supported in part by the National Natural Science Foundation of China (NSFC) under Grant 92373103, 62204271, 62334014, and in part by the Opening Project of Key Laboratory of Automotive Chip Testing and Evaluation, State Administration for Market Regulation under Grant QCXP202501.

REFERENCES

- [1] J. Alsop, M. S. Orr, B. M. Beckmann, and D. A. Wood, "Lazy Release Consistency for GPUs," in *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2016, pp. 1–14.
- [2] A. Arunkumar, E. Bolotin, B. Cho, U. Milic, E. Ebrahimi, O. Villa, A. Jaleel, C.-J. Wu, and D. Nellans, "MCM-GPU: Multi-Chip-Module GPUs for Continued Performance Scalability," *ACM SIGARCH Computer Architecture News*, vol. 45, no. 2, pp. 320–332, 2017.
- [3] A. Arunkumar, E. Bolotin, D. Nellans, and C.-J. Wu, "Understanding the Future of Energy Efficiency in Multi-Module GPUs," in *2019 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2019, pp. 519–532.
- [4] S. Che, B. M. Beckmann, S. K. Reinhardt, and K. Skadron, "Pannotia: Understanding Irregular GPGPU Graph Applications," in *2013 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 2013, pp. 185–195.
- [5] S. Che, M. Boyer, J. Meng, D. Tarjan, J. W. Sheaffer, S.-H. Lee, and K. Skadron, "Rodinia: A Benchmark Suite for Heterogeneous Computing," in *2009 IEEE International Symposium on Workload Characterization (IISWC)*, 2009.
- [6] S. Chetlur, C. Woolley, P. Vandermersch, J. Cohen, J. Tran, B. Catanzaro, and E. Shelhamer, "cudnn: Efficient Primitives for Deep Learning," *arXiv preprint arXiv:1410.0759*, 2014.
- [7] Y. H. Chou, C. Ng, S. Cattell, J. Intan, M. D. Sinclair, J. Devietti, T. G. Rogers, and T. M. Aamodt, "Deterministic Atomic Buffering," in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2020, pp. 981–995.
- [8] P. Dalmia, R. S. Kumar, and M. D. Sinclair, "CPElide: Efficient Multi-Chiplet GPU Implicit Synchronization," in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2024, pp. 700–717.
- [9] P. Dalmia, R. Mahapatra, J. Intan, D. Negrut, and M. D. Sinclair, "Improving the Scalability of GPU Synchronization Primitives," *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, no. 1, pp. 275–290, 2022.
- [10] P. Dalmia, R. Mahapatra, and M. D. Sinclair, "Only buffer When You Need to: Reducing on-Chip GPU Traffic with Reconfigurable Local Atomic Buffers," in *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2022, pp. 676–691.
- [11] S. Durvasula, A. Zhao, F. Chen, R. Liang, P. K. Sanjaya, Y. Guan, C. Giannoula, and N. Vijaykumar, "ARC: Warp-level Adaptive Atomic Reduction in GPUs to Accelerate Differentiable Rendering," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, 2025, pp. 64–83.
- [12] A. ElTantawy and T. M. Aamodt, "Mimd synchronization on simt architectures," in *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2016, pp. 1–14.
- [13] A. ElTantawy and T. M. Aamodt, "Warp scheduling for fine-grained synchronization," in *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2018, pp. 375–388.
- [14] Y. Feng, S. Na, H. Kim, and H. Jeon, "Barre Chord: Efficient Virtual Memory Translation for Multi-Chip-Module GPUs," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2024, pp. 834–847.
- [15] Y. Fu, E. Bolotin, N. Chatterjee, D. Nellans, and S. W. Keckler, "GPU Domain Specialization via Composable on-Package Architecture," *ACM*

- Transactions on Architecture and Code Optimization (TACO)*, vol. 19, no. 1, pp. 1–23, 2021.
- [16] B. A. Hechtman, S. Che, D. R. Hower, Y. Tian, B. M. Beckmann, M. D. Hill, S. K. Reinhardt, and D. A. Wood, “QuickRelease: A Throughput-Oriented Approach to Release Consistency on GPUs,” in *2014 IEEE 20th International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2014, pp. 189–200.
 - [17] B. A. Hechtman and D. J. Sorin, “Exploring Memory Consistency for Massively-Threaded Throughput-Oriented Processors,” in *Proceedings of the 40th Annual International Symposium on Computer Architecture*, 2013, pp. 201–212.
 - [18] D. R. Hower, B. A. Hechtman, B. M. Beckmann, B. R. Gaster, M. D. Hill, S. K. Reinhardt, and D. A. Wood, “Heterogeneous-Race-Free Memory Models,” in *Proceedings of the 19th international conference on Architectural support for programming languages and operating systems*, 2014, pp. 427–440.
 - [19] N. Jiang, D. U. Becker, G. Micheliogiannakis, J. Balfour, B. Towles, D. E. Shaw, J. Kim, and W. J. Dally, “A Detailed and Flexible Cycle-Accurate Network-on-Chip Simulator,” in *2013 IEEE international symposium on performance analysis of systems and software (ISPASS)*. IEEE, 2013, pp. 86–96.
 - [20] H. Jooybar, W. W. Fung, M. O’Connor, J. Devietti, and T. M. Aamodt, “GPUDet: A Deterministic GPU Architecture,” in *Proceedings of the eighteenth international conference on Architectural support for programming languages and operating systems*, 2013, pp. 1–12.
 - [21] V. Kandiah, S. Peverelle, M. Khairy, J. Pan, A. Manjunath, T. G. Rogers, T. M. Aamodt, and N. Hardavellas, “AccelWatch: A Power Modeling Framework for Modern GPUs,” in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021, pp. 738–753.
 - [22] A. Kannan, N. E. Jerger, and G. H. Loh, “Exploiting Interposer Technologies to Disintegrate and Reintegrate Multicore Processors,” *Ieee Micro*, vol. 36, no. 3, pp. 84–93, 2016.
 - [23] S. Keckler, “Life After Dennard and How I Learned to Love the Picojoule,” *Keynote at MICRO*, 2011.
 - [24] M. Khairy, Z. Shen, T. M. Aamodt, and T. G. Rogers, “Accel-sim: An Extensible Simulation Framework for Validated GPU Modeling,” in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2020, pp. 473–486.
 - [25] K. Koukos, A. Ros, E. Hagersten, and S. Kaxiras, “Building Heterogeneous Unified Virtual Memories (UVMs) Without the Overhead,” *ACM Transactions on Architecture and Code Optimization (TACO)*, vol. 13, no. 1, pp. 1–22, 2016.
 - [26] A. Lavin and S. Gray, “Fast Algorithms for Convolutional Neural Networks,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 4013–4021.
 - [27] J. M. Mellor-Crummey and M. L. Scott, “Algorithms for Scalable Synchronization on Shared-Memory Multiprocessors,” *ACM Transactions on Computer Systems (TOCS)*, vol. 9, no. 1, pp. 21–65, 1991.
 - [28] D. Merrill, “NVIDIA CUB Library,” <https://nvlabs.github.io/cub/>, 2020.
 - [29] U. Milic, O. Villa, E. Bolotin, A. Arunkumar, E. Ebrahimi, A. Jaleel, A. Ramirez, and D. Nellans, “Beyond the socket: NUMA-aware GPUs,” in *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture*, 2017, pp. 123–135.
 - [30] G. E. Moore, “Cramming More Components onto Integrated Circuits,” *Proceedings of the IEEE*, vol. 86, no. 1, pp. 82–85, 1998.
 - [31] M. A. O’Neil and M. Burtcher, “Microarchitectural Performance Characterization of Irregular GPU Kernels,” in *2014 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 2014, pp. 130–139.
 - [32] [Online], “llm.c,” <https://github.com/karpathy/llm.c>.
 - [33] [Online], “Simple-vgg16-cu,” <https://github.com/rezaisajjad/simple-vgg16-cu>.
 - [34] [Online], “Nvidias Next Generation CUDA Compute-Architecture: Kepler gk110,” <https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/tesla-product-literature/NVIDIA-Kepler-GK110-GK210-Architecture-Whitepaper.pdf>, 2012.
 - [35] [Online], “TOP500 Shows Growing Momentum for Accelerators,” <https://insidehpc.com/2015/11/top500-shows-growing-momentum-for-accelerators/>, 2015.
 - [36] [Online], “NVIDIA Tesla P100 Architecture,” <https://images.nvidia.com/content/pdf/tesla/whitepaper/pascal-architecture-whitepaper.pdf>, 2016.
 - [37] [Online], “NVIDIA Tesla V100 Architecture,” <https://images.nvidia.com/content/volta-architecture/pdf/volta-architecture-whitepaper.pdf>, 2017.
 - [38] M. S. Orr, S. Che, A. Yilmazer, B. M. Beckmann, M. D. Hill, and D. A. Wood, “Synchronization using Remote-Scope Promotion,” *ACM SIGARCH Computer Architecture News*, vol. 43, no. 1, pp. 73–86, 2015.
 - [39] S. Pal, D. Petrisco, M. Tomei, P. Gupta, S. S. Iyer, and R. Kumar, “Architecting Waferscale Processors-a GPU Case Study,” in *2019 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2019, pp. 250–263.
 - [40] J. W. Poulton, W. J. Dally, X. Chen, J. G. Eyles, T. H. Greer, S. G. Tell, J. M. Wilson, and C. T. Gray, “A 0.54 pJ/b 20 Gb/s Ground-Referenced Single-Ended Short-Reach Serial Link in 28 nm CMOS for Advanced Packaging Applications,” *IEEE Journal of Solid-State Circuits*, vol. 48, no. 12, pp. 3206–3218, 2013.
 - [41] J. Power, A. Basu, J. Gu, S. Puthoor, B. M. Beckmann, M. D. Hill, S. K. Reinhardt, and D. A. Wood, “Heterogeneous System Coherence for Integrated CPU-GPU Systems,” in *Proceedings of the 46th Annual IEEE/ACM International Symposium on Microarchitecture*, 2013, pp. 457–467.
 - [42] B. Pratheek, N. Jawalkar, and A. Basu, “Designing Virtual Memory System of MCM GPUs,” in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2022, pp. 404–422.
 - [43] X. Ren, D. Lustig, E. Bolotin, A. Jaleel, O. Villa, and D. Nellans, “Hmg: Extending Cache Coherence Protocols Across Modern Hierarchical Multi-GPU Systems,” in *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2020, pp. 582–595.
 - [44] H. SeyyedAghaei, M. Naderan-Tahan, M. Jahre, and L. Eeckhout, “Memory-Centric MCM-GPU Architecture,” *IEEE Computer Architecture Letters*, vol. 24, no. 1, pp. 101–104, 2025.
 - [45] Y. S. Shao, J. Clemons, R. Venkatesan, B. Zimmer, M. Fojtik, N. Jiang, B. Keller, A. Klinefelter, N. Pinckney, P. Raina *et al.*, “Simba: Scaling Deep-Learning Inference with Multi-Chip-Module-Based Architecture,” in *Proceedings of the 52nd annual IEEE/ACM international symposium on microarchitecture*, 2019, pp. 14–27.
 - [46] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” *arXiv preprint arXiv:1409.1556*, 2014.
 - [47] M. D. Sinclair, J. Alsop, and S. V. Adve, “Efficient GPU Synchronization Without Scopes: Saying No to Complex Consistency Models,” in *Proceedings of the 48th International Symposium on Microarchitecture*, 2015, pp. 647–659.
 - [48] M. D. Sinclair, J. Alsop, and S. V. Adve, “HeteroSync: A Benchmark Suite for Fine-Grained Synchronization on Tightly Coupled GPUs,” in *2017 IEEE international symposium on workload characterization (IISWC)*. IEEE, 2017, pp. 239–249.
 - [49] I. Singh, A. Shriraman, W. W. Fung, M. O’Connor, and T. M. Aamodt, “Cache Coherence for GPU Architectures,” in *2013 IEEE 19th International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2013, pp. 578–590.
 - [50] A. Snell and L. Segervall, “HPC Application Support for GPU Computing,” *Intersect365 Research*, 2017.
 - [51] K. Wang, D. Fussell, and C. Lin, “Fast fine-grained global synchronization on gpus,” in *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2019, pp. 793–806.
 - [52] S. Zhang, M. Naderan-Tahan, M. Jahre, and L. Eeckhout, “SAC: Sharing-aware Caching in Multi-Chip GPUs,” in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–13.
 - [53] X. Zhang, G. Zhang, L. Wang, S. Zhang, and X. Zhao, “AdCoalescer: An Adaptive Coalescer to Reduce the Inter-Module Traffic in MCM-GPUs,” in *Proceedings of the 53rd International Conference on Parallel Processing*, 2024, pp. 1001–1011.
 - [54] Y. Zhang, M. Wang, W. Wang, Y. Mai, H. Huang, and Z. Yu, “Atomic Cache: Enabling Efficient Fine-Grained Synchronization with Relaxed Memory Consistency on GPGPUs Through In-Cache Atomic Operations,” in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2024, pp. 671–685.
 - [55] X. Zhao, G. Zhang, L. Wang, S. Zhang, and H. Dai, “NearFetch: Saving Inter-Module Bandwidth in Many-Chip-Module GPUs,” in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2025, pp. 1693–1706.